

Strong computer-science, algorithms, and research problem-solving background with hands-on C++ and Python, LLVM, and compiler/runtime engineering. I work in a hypothesis → experiment → validation loop using exact oracles, randomized/metamorphic tests, reproducible harnesses and benchmarks, counterexample analysis, and performance investigation. I work with JIT compilation, vectorization, x86-64 assembly, and generated/disassembled machine-code analysis

Algorithms & competitions

HSE Vysshaya Proba prize-winner; overall winner of the MEPhI Junior competition in 2025 and 2026; Grand Prize at the Baltic science and engineering competition

Hypothesis → experiment

Exact oracles, randomized/metamorphic testing, adversarial counterexamples, reproducible harnesses, and benchmarks

C++ / low-level performance

C++23, LLVM, JIT, vectorization, x86-64 assembly, generated/disassembled machine-code analysis, and benchmarks

EXPERIENCE		SKILLS
July 1 — August 31, 2026	MCST — compiler engineering intern - Implemented an LLVM loop-invariant code-motion pass in C++; reasoned about loop invariance, side effects, memory access, speculative safety, and loop structure - Built a Python verification harness comparing execution behavior before and after optimization and checking expected transformation / no-transformation cases	Programming C++23, Python, C17, C#, Rust Algorithms & CS data structures, graph algorithms, program analysis, CFG/SSA, compiler optimizations Low-level & performance LLVM, JIT/CIL, x86-64 assembly, vectorization, compiler optimizations, generated/disassembled machine-code analysis, benchmarking Experiments & tooling hypothesis formulation and validation, exact oracles, differential/metamorphic/randomized testing, Python harnesses, Linux, Git, CMake
2026	PS-form Memory Dependence Analyzer - Built conservative analysis checked by an independent exact oracle on small domains, randomized/metamorphic cases, and reproducible counterexamples - Built an exact oracle for small domains, metamorphic/randomized checks, and reproducible wrong-answer/time-limit counterexamples	
2024 — present	UniversalToolchain — creator and primary developer - Independently designed and developed a complex compiler/language system with intermediate representations, SSA/optimizations, and interpreter/CIL backends - Use benchmarking, JIT/CIL execution, and generated/disassembled-code analysis to investigate implementation behavior and performance	EDUCATION HSE University — Software Engineering, 2026–2030 RECOGNITION First-degree diploma and the Grand Prize “Perfection as Hope” for UniversalToolchain. Moscow / remote

PROJECTS

PS-form Memory Dependence Analyzer

Conservative yes/no/maybe results checked by an independent exact oracle on small domains, randomized/metamorphic tests, and reproducible counterexamples.

UniversalToolchain

LanguageRuntime validates and materializes the exact planned package/component graph without re-planning; plan-determinism/exact-binding checks, ownership guards, and interpreter/CIL parity protect the contract.

x86-64 Codegen & Register Allocation Lab

A reference interpreter and differential execution check generated-code semantics; the allocator assignment contract is verified independently.